

SHOPPING CART BY VECTOR DS

Team Members:

- Aya Mohamed
- Bosina Saad
- Habiba Ahmed
- Nadia Kamel

By Dr: LAMIA.ALREFAAI

Shopping Cart based on vector Data Structure

Overview

In this project, we will be creating a shopping cart application using a vector data structure in a programming language C++. The goal is to demonstrate how a vector can be used to store and manage items in a shopping cart by assembly Mips language.

Vector Data Structure

A vector is a dynamic array data structure that can grow and shrink in size as needed. Unlike a static array which has a fixed size, a vector can automatically allocate more memory as elements are added to it. This makes vectors very flexible and convenient for storing collections of items.

Main Functions:

- Main
- Signup
- Login
- Shop_initialize
- purchase
- StartShopping
- SearchByshop
- Shop_selected
- SearchByItem
- Item_selecte
- Cart
- Order
- Exit_program
- Vector Main Functions
 1. Push_back: Add element at the end of vector.
 2. Pop_back: Delete element from the end of vector.
 3. Clear: Delete all element of the vector
 4. Vector_construct: create the vector and save its dynamic size in memory.
 5. Print_puch:print vector.

Explain Code

Prompts User Interaction

Description:

These strings are essential for user interaction and providing feedback in the e-commerce program. Each serves a specific purpose, whether it's guiding the user through the signup process, confirming their actions, or alerting them to errors.

- Prompts for Welcome: Welcome messages for the e-commerce website Prompts for signup, login, and exit options Messages for thanking the user for visiting or cancelling an item.
- Prompts for entering user information: during signup (name, email, password, mobile number, age) and login (password) Prompts for entering counts of items.
- Login and Signup Pages: Messages indicating the start of the signup and login processes.
- Login Status Messages: Messages for successful and unsuccessful login attempts, including invalid password and account non-existence.
- Order-related Prompts: Prompt to confirm if the user wishes to proceed with an order Message confirming the order and indicating that the item is on the way.
- Error Messages: Message indicating account already exists or account creation success.
- Search Options and Item Selection: Messages related to search options (search by shop or item). Prompts for choosing shops and items. Prompt to select items in the cart.
- Cart Management: Header for the cart display. Prompt to remove the last item from the cart. Options for exiting or continuing with the cart.
- Miscellaneous: Strings for spacing, newline characters, and messages indicating the removal of the last item from the cart.

Algorithm

- Declaration:
 1. Set variable name.
 2. Data type .asciiz .
 3. Write text between double quates “ ”.

Definition of variables

Description

The provided code snippet declares several variables with allocated space for storing different types of data such as names, emails, passwords, mobile numbers, usernames, and an age value. These variables are reserved in memory to be used for storing user information in a program. And use Word for storing data in 4 byte in memory.

```
# Add variables here
t_name:      .space 64
name:        .space 64
t_email:     .space 64
t_password1: .space 12
pass:        .space 12
t_mobile:    .space 12
user_name:   .space 64
t_age:       .word 0
```

Algorithm

- Set variable name.
- Determine size of them using .word or .space and the size of memory.

Definition of Struct

Description

his code snippet is defining a structure called "Shops" that contains various labels, each representing a shop or brand name, followed by its associated products or items. The labels are defined using the .ascii directive, which allocates space for the null-terminated strings in memory.

```
# Structure: Shops
.Puma:
    .ascii "Puma"
.puma_sneaker_V2:
    .ascii "1)puma sneaker V2"
.mens_dryflex:
    .ascii "2)mens dryflex"
.one8x_puma:
    .ascii "3)one8x puma"
.USA:
    .ascii "\n2)USA"
.US_polo_Tshirt:
    .ascii "4)US polo Tshirt"
.Shoes:
    .ascii "5)Shoes"
.shirt:
    .ascii "6)shirt"
.WROGN:
    .ascii "\n3)WROGN"
.Mens_Watch:
    .ascii "7)Mens Watch"
.mens_solid_jacket:
    .ascii "8)mens solid jacket"
.casual_Tshirt:
    .ascii "9)casual T-shirt"
```

Algorithm

- Allocate Memory for the Struct based on struct size
- Create a Struct Variable
- Assigning Values to Struct Members

Main Function

Description

sets up the initial flow of the program, initializes shops and purchases, prompts the user for input, and directs the program flow based on the user's choice. It follows a structured approach to handle different user actions within the program.

Algorithm

- calls the shop_initialize and purchase functions using jal (jump and link) instructions.
- displays prompts using syscall 4 to print strings
- reads an integer input from the user using syscall 5 and stores it in register \$v0.
- value in register \$v0 is moved to register \$t1.
- The program then enters a switch statement labeled switch based on the value in register \$t1.
- Depending on the value of \$t1, the program branches to different sections of code (signup, login, exit_program) using beq (branch if equal) instructions.
- If none of the expected values are found, it displays another prompt and jumps back to the main label to restart the process

The Code

The OutPut

```
.globl main
main:

jal shop_initialize
jal purchase

    li $v0, 4
    la $a0, prompt1
    syscall

    li $v0, 4
    la $a0, prompt2
    syscall

    li $v0, 5
    syscall
    move $t1, $v0

switch:
beq $t1, 1, signup
beq $t1, 2, login
beq $t1, 3, exit_program
li $v0, 4
la $a0, prompt4
syscall
j main
```

```
****Welcome to the E-commerce Website****
```

```
1) Signup
2) Login
3) Exit
```

```
Please Enter Your choice : 1
```

Signup Function

Description

the signup process for a user in a program. It prompts the user for their name, email, password, and mobile number, stores the input in registers, and then prints a success message before jumping to the login process.

Algorithm

- printing several prompts to the console using syscall 4.
- reads user input using syscall 8.
 1. t_name: The user's name, with a maximum length of 64 characters.
 2. t_email: The user's email address, with a maximum length of 64 characters.
 3. t_password1: The user's password, with a maximum length of 12 characters.
 4. t_mobile: The user's mobile number, with a maximum length of 12 characters.
 5. Store user input in registers: The code stores the user input in registers \$t3, \$t4, and \$t5 for later
- Print a success message: After reading all the user data, the code prints a success message using syscall 4, indicating that the user's account has been created.
- Jump to the login label

The Code

```
signup:
    li $v0, 4
    la $a0, prompt5
    syscall

    li $v0, 4
    la $a0, prompt6
    syscall
    li $v0, 8
    la $a0, t_name
    li $a1, 64
    syscall

    la $t3, t_name
    li $v0, 4
    la $a0, prompt7
    syscall
    li $v0, 8
    la $a0, t_email
    li $a1, 64
    syscall

    li $v0, 4
    la $a0, prompt13
    syscall
    li $v0, 8
    la $a0, t_password1
    li $a1, 12
    syscall
    la $t4, t_password1

    li $v0, 4
    la $a0, prompt9
    syscall
    li $v0, 8
    li $v0, 5
    syscall
    move $t5, $v0

    li $v0, 4
    la $a0, account_created_str
    syscall
    j login
```

The OutPut

```
****Welcome to the E-commerce Website****

1) Signup
2) Login
3) Exit

Please Enter Your choice : 1

*****Welcome to Signup Page*****

Enter Your name : aya
Enter Your Email : aya@.com
Enter Your Password: 123456***
Enter Your Mobile Number : 0122657
Enter Your Age : 21

Account successfully created!
```

Login Function

Description

this function snippet facilitates the login process by verifying the user's identity through name and password by compare them with the user name and password those taken at signup function, and controlling the program flow based on the login outcome.

Algorithm

- printing prompts to the console and reading user input for their name and password.
- compares the user's name with a stored name (t_name) character by character using a loop. If the names match, it proceeds to check the password.
- The password comparison loop is similar to the name comparison loop, but it checks the user's password with a stored password (t_password1).
- If the password matches, the program prints a success message and displays the user's name.
- If the password or name does not match, it prints an error message and returns to the main menu.

The Code

login:

```
li $v0, 4
la $a0, prompt11
syscall

li $v0, 4
la $a0, prompt6
syscall

li $v0, 8
la $a0, name
li $a1, 64
syscall

li $v0, 4
la $a0, prompt13
syscall

li $v0, 8
la $a0, pass
li $a1, 12
syscall

la $t0, name
la $t1, t_name

compare_loop:
    lb $t2, 0($t0)
    lb $t3, 0($t1)

    beq $t2, $zero, strings_equal
    beq $t3, $zero, strings_not_equal
    bne $t2, $t3, strings_not_equal
    addi $t0, $t0, 1
    addi $t1, $t1, 1
    j compare_loop

strings_equal:
    j pass_check
```

```
strings_not_equal:
    li $v0, 4
    la $a0, prompt18
    syscall
    j main
```

pass_check:

```
la $t0, pass
la $t1, t_password1
```

Compare_loop:

```
lb $t2, 0($t0)
lb $t3, 0($t1)

beq $t2, $zero, Strings_equal
beq $t3, $zero, Strings_not_equal
bne $t2, $t3, Strings_not_equal
addi $t0, $t0, 1
addi $t1, $t1, 1
j Compare_loop
```

Strings_equal:

```
li $v0, 4
la $a0, prompt16
syscall

li $v0, 4
la $a0, t_name
syscall

li $v0, 4
la $a0, prompt17
syscall
j StartShopping
```

Strings_not_equal:

```
li $v0, 4
la $a0, prompt18
syscall
j main
```

The Output

```
*****Welcome to Login Page*****  
  
Enter Your name : aya  
Enter Your Password: 123456***  
  
Welcome aya  
, You are successfully logged in
```

Shop_initialization Function

Description

this function snippet initializes a shopping cart system by constructing a vector and populating it with a list of products from different brands along with their prices.

Algorithm

- allocating space on the stack (addi \$sp, \$sp, -8) and saving the return address (sw \$ra, 4(\$sp)).
- Vector Construction: constructs a vector of size 5 (li \$a0, 5; jal Vector_constructor_int_n) and assigns it to a variable Mole.
- initializes a list of products with their names and prices. Each product is represented by a string and a price stored in memory.
- The list includes products from different brands such as Puma, USA, and WROGN, along with their respective prices.
- After initializing the products, the code restores the return address (lw \$ra, 4(\$sp)) and returns from the function (addi \$sp, \$sp, 8; jr \$ra)

The Code

```
shop_initialize:
    addi $sp, $sp, -8
    sw $ra, 4($sp)

    li $a0, 5
    jal Vector_constructor_int_n

    la $t0, Mole
    li $t1, 1

    la $t2, .Puma
    sw $t2, 0($t0)

    la $t2, .puma_sneaker_V2
    sw $t2, 4($t0)

    la $t2, .mens_dryflex
    sw $t2, 8($t0)

    la $t2, .one8x_puma
    sw $t2, 12($t0)

    li $t2, 3499
    sw $t2, 16($t0)

    li $t2, 2999
    sw $t2, 20($t0)

    li $t2, 5999
    sw $t2, 24($t0)

    addi $t0, $t0, 32
    li $t1, 2

    la $t2, .USA
    sw $t2, 0($t0)

    la $t2, .US_polo_Tshirt
    sw $t2, 4($t0)

    la $t2, .Shoes
    sw $t2, 8($t0)

    la $t2, .shirt
    sw $t2, 12($t0)

    li $t2, 999
    sw $t2, 16($t0)

    li $t2, 2000
    sw $t2, 20($t0)

    li $t2, 1499
    sw $t2, 24($t0)

    addi $t0, $t0, 32
    li $t1, 3

    la $t2, .WROGN
    sw $t2, 0($t0)

    la $t2, .Mens_Watch
    sw $t2, 4($t0)

    la $t2, .mens_solid_jacket
    sw $t2, 8($t0)

    la $t2, .casual_Tshirt
    sw $t2, 12($t0)

    li $t2, 2000
    sw $t2, 16($t0)

    li $t2, 2239
    sw $t2, 20($t0)

    li $t2, 799
    sw $t2, 24($t0)
    lw $ra, 4($sp)
    addi $sp, $sp, 8
    jr $ra
```

StartShopping Function

Description

this function snippet initiates the shopping process by presenting menu options of searchByShop or searchByItem or exit_program , reading the user's choice, and directing the program flow based on the user's selection using a switch case structure.

Algorithm

- Displaying menu options to the user using syscall 4 to print strings. It presents options for searching by shop, searching by item, and exiting the program.
- prompts the user to enter their choice using syscall 4 to display a prompt and syscall 5 to read the user's input. The choice is stored in register \$t0.
- uses a switch case structure to handle the user's choice:
 1. If the user chooses option 1, it branches to the searchByShop section of the code.
 2. If the user chooses option 2, it branches to the searchByItem section of the code.
 3. If the user chooses option 3, it branches to the exit_program section to terminate the program.
 4. If the user enters an invalid choice, it displays an error message and returns to the start of the shopping process.

The Code

```
StartShopping:
    # Print the menu options
    li $v0, 4
    la $a0, search_option_str
    syscall

    li $v0, 4
    la $a0, search_by_shop_str
    syscall

    li $v0, 4
    la $a0, search_by_item_str
    syscall

    li $v0, 4
    la $a0, exit_str
    syscall

    li $v0, 4
    la $a0, choice_str
    syscall

    # Read the user's choice
    li $v0, 5
    syscall
    move $t0, $v0

    # Perform the switch case
    beq $t0, 1, searchByShop
    beq $t0, 2, searchByItem
    beq $t0, 3, exit_program

    li $v0, 4
    la $a0, error_invalid_name
    syscall
    j StartShopping
```

The OutPut

```
We provide two ways of search:
1) Search by Shop
2) Search by Item
3) Exit
```

```
Please Enter your choice : 1
```

SearchByshop Function

Description

this function snippet allows the user to search for products by shop. It displays the available shops, reads the user's choice, validates it, and directs the program flow based on the user's selection.

Algorithm

- Print Shop Data
 1. enters a loop labeled print_loop that iterates through the shop data stored in the Mole variable.
 2. For each shop, it loads the shop name address into register \$a0 and prints it using syscall 4.
 3. moves to the next shop data by incrementing the address in \$t0 by 32 bytes and the loop counter \$t1.
 4. The loop continues until all shops have been printed (3 iterations).
- After displaying the shop options, the code prompts the user to enter their choice using syscall 4 to print a string and syscall 5 to read the user's input, which is stored in register \$v0.
- checks if the user's choice is valid by comparing it with the minimum and maximum valid choices (1 and 4, respectively).
- If the choice is invalid, it displays an error message and jumps back to the searchByShop label to prompt the user again.
- If the user chooses option 4, it jumps back to the StartShopping label to restart the shopping process.
- If the user chooses a valid shop (options 1-3), it moves the choice to register \$a0 and jumps to the Shop_selected label to handle the selected shop.
- If the user chooses to exit, it jumps to the exit_program label to terminate the program

The Code

```
searchByShop:
    li $v0, 4
    la $a0, choose_shop_str
    syscall

print_shop_data:
    la $t0, Mole
    li $t1, 1

print_loop:

    lw $a0, 0($t0)
    li $v0, 4
    syscall
    # Move to next Mole element
    addi $t0, $t0, 32
    addi $t1, $t1, 1
    blt $t1, 4, print_loop

    li $v0, 4
    la $a0, shop_exit_str
    syscall
    li $v0, 5
    syscall

li $t0, 0
ble $v0, $t0, invalid_choice
li $t0, 4
bgt $v0, $t0, invalid_choice

beq $v0, $t0, StartShopping
move $a0, $v0
j Shop_selected
j exit_program
invalid_choice:

li $v0, 4
la $a0, prompt4
syscall
j searchByShop
```

The OutPut

```
Please Choose the Shop
```

```
1) Puma
2) USA
3) WROGN
4) Exit
```

```
Please select the shop 1
```

Shop_selected Function

Description

this function snippet handles the selection of items from different shops, displays item details, calculates prices based on user input, and manages the flow of the shopping process within the program.

Algorithm

- starts by processing the user's choice of shop.
- checks the user's choice and directs the program flow based on the selected shop (Puma, USA, or others).
- For the selected shop, the code enters a loop to print the items available along with their prices.
- loads item names and prices from memory and prints them to the console using syscall 4 for strings and syscall 1 for integers.
- prompts the user to choose an item or perform specific actions like adding items to the cart, returning to the main menu, or entering the quantity of items.
- calculates the total price based on the selected item, quantity, and price by multiplying the item price by the quantity and adds it to a running total.

- If the user enters an invalid choice or quantity, the code displays an error message and prompts the user again.
- based on the user's choices, allowing them to select items, add them to the cart, return to the main menu, or handle errors gracefully.

The OutPut

```

Please Enter the Count of 3
1)puma sneaker V2      --> 3499
2)mens dryflex  --> 2999
3)one8x puma  --> 5999

4) Cart
5) Exit

Please Enter your choice : _____
  
```

```

4)US polo Tshirt      --> 999
5)Shoes      --> 2000
6)shirt      --> 1499

4) Cart
5) Exit

Please Enter your choice : _____
  
```

```

Please select the shop 3
7)Mens Watch      --> 2000
8)mens solid jacket  --> 2239
9)casual T-shirt  --> 799

4) Cart
5) Exit

Please Enter your choice :
-
  
```

The Code of Shop selected Function

```
Shop_selected:
addi $s1,$a0,0
    la $s0, Mole
    addi $t0,$s0,0
    li $t1, 1          # Index for Mole[1]

    li $t6,1
    beq $s1,$t6,puma
    li $t6,2
    beq $s1,$t6,usa
    addi $t0, $t0, 64
    addi $t1, $t1, 2
    printw:
    lw $a0, 4($t0)      # Load item1
    li $v0, 4
    syscall
    li $v0, 4
    la $a0, Space
    syscall

    lw $a0, 16($t0)     # Load first price
    li $v0, 1
    syscall
    li $v0, 4
    la $a0, newline
    syscall

    lw $a0, 8($t0)      # Load item2
    li $v0, 4
    syscall

    li $v0, 4
    la $a0, Space
    syscall

    lw $a0, 20($t0)     # Load second price
    li $v0, 1
    syscall
    li $v0, 4
    la $a0, newline
    syscall

    lw $a0, 12($t0)     # Load item3
    li $v0, 4
    syscall
    li $v0, 4
    la $a0, Space
    syscall

    lw $a0, 24($t0)     # Load third price
    li $v0, 1
    syscall
    li $v0, 4
    la $a0, newline
    syscall

    la $a0, cart_exit_st
    li $v0, 4
    syscall
    li $v0, 4
    la $a0, newline
    syscall

    la $a0, choice_str
    li $v0, 4
    syscall
    li $v0, 4
    la $a0, newline
    syscall
    li $v0, 5
    syscall
    addi $t7,$v0,0#item
    blt $v0,1,invw
    bgt $v0,5,invw
    beq $v0,4, cart
    beq $v0,5, searchByShop
    la $a0, prompt22
    li $v0, 4
    syscall
    li $v0, 5
    syscall
    addi $t6,$v0,0#n
```

```

        beq $t7, 1, calculate_item1w
        beq $t7, 2, calculate_item2w
        j calculate_item3w

calculate_item1w:
    lw $t5, 16($t0) # Load first price # load Mole[shop_choice].first into $t1
    mul $t5, $t5, $t6 # multiply n with Mole[shop_choice].first
    add $s4, $s4, $t5
    lw $a0, 4($t0) #item
    lw $a1, 16($t0) #price
    move $a2, $t6 # n
    jal push_back
    j printw

calculate_item2w:
    lw $t5, 20($t0) # Load second price
    mul $t5, $t5, $t6 # multiply n with Mole[shop_choice].first
    add $s4, $s4, $t5
    lw $a0, 8($t0) #item
    lw $a1, 20($t0) #price
    move $a2, $t6 # n
    jal push_back
    j printw

calculate_item3w:
    lw $t5, 24($t0) # Load third price
    mul $t5, $t5, $t6 # multiply n with Mole[shop_choice].first
    add $s4, $s4, $t5
    lw $a0, 12($t0) #item
    lw $a1, 24($t0) #price
    move $a2, $t6 # n
    jal push_back
    j printw

inrw:
    la $a0, prompt4
    li $v0, 4
    syscall
    j printw

li $v0, 4
    la $a0, newline
    syscall

    la $a0, cart_exit_st
    li $v0, 4
    syscall

li $v0, 4
    la $a0, newline
    syscall

    la $a0, choice_str
    li $v0, 4
    syscall
    li $v0, 4
    la $a0, newline
    syscall
    li $v0, 5
    syscall
    addi $t7, $v0, 0 #item
    blt $v0, 1, invp
    bgt $v0, 5, invp

    beq $v0, 4, cart
    beq $v0, 5, searchByShop
    la $a0, prompt22
    li $v0, 4
    syscall
    li $v0, 5
    syscall
    addi $t6, $v0, 0 #n
    beq $t7, 1, calculate_item1p
    beq $t7, 2, calculate_item2p
    j calculate_item3p

```

```

puma:

    lw $a0, 4($t0)    # Load item1
    li $v0, 4
    syscall
    li $v0, 4
    la $a0, Space
    syscall

    lw $a0, 16($t0)   # Load first price
    li $v0, 1
    syscall
    li $v0, 4
    la $a0, newline
    syscall

    lw $a0, 8($t0)    # Load item2
    li $v0, 4
    syscall
    li $v0, 4
    la $a0, Space
    syscall

    lw $a0, 20($t0)   # Load second price
    li $v0, 1
    syscall
i $v0, 4
    la $a0, newline
    syscall
    lw $a0, 12($t0)   # Load item3
    li $v0, 4
    syscall
    li $v0, 4
    la $a0, Space
    syscall

    lw $a0, 24($t0)   # Load third price
    li $v0, 1
    syscall

```

```

calculate_item1:
    lw $t5, 16($t0)   # Load first price    # load Mole[shop_choice].first into $t1
    mul $t5, $t5, $t6   # multiply n with Mole[shop_choice].first
    add $s4,$s4,$t5
    lw $a0, 4($t0) #item
    lw $a1, 16($t0) #price
    move $a2, $t6 # n
    jal push_back
    j puma
calculate_item2:
    lw $t5, 20($t0)   # Load second price
    mul $t5, $t5, $t6   # multiply n with Mole[shop_choice].first
    add $s4,$s4,$t5
    lw $a0, 8($t0) #item
    lw $a1, 20($t0) #price
    move $a2, $t6 # n
    jal push_back
    j puma
calculate_item3:
    lw $t5, 24($t0)   # Load third price
    mul $t5, $t5, $t6   # multiply n with Mole[shop_choice].first
    add $s4,$s4,$t5
    lw $a0, 12($t0) #item
    lw $a1, 24($t0) #price
    move $a2, $t6 # n
    jal push_back
    j puma
inrp:
    la $a0,prompt4
    li $v0,4
    syscall
    j puma

```

```

usa:
    addi $t0, $t0, 32
    addi $t1, $t1, 1
    printu:
    lw $a0, 4($t0)      # Load item1
    li $v0, 4
    syscall
    li $v0, 4
    la $a0, Space
    syscall

    lw $a0, 16($t0)     # Load first price
    li $v0, 1
    syscall
    li $v0, 4
    la $a0, newline
    syscall

    lw $a0, 8($t0)      # Load item2
    li $v0, 4
    syscall
    li $v0, 4
    la $a0, Space
    syscall

    lw $a0, 20($t0)     # Load second price
    li $v0, 1
    syscall
    li $v0, 4
    la $a0, newline
    syscall

    lw $a0, 12($t0)     # Load item3
    li $v0, 4
    syscall
    li $v0, 4
    la $a0, Space
    syscall

    lw $a0, 24($t0)     # Load third price
    li $v0, 1
    syscall

    li $v0, 4
    la $a0, newline
    syscall

    la $a0, cart_exit_st
    li $v0, 4
    syscall

    li $v0, 4
    la $a0, newline
    syscall

    la $a0, choice_str
    li $v0, 4
    syscall

    li $v0, 4
    la $a0, newline
    syscall
    li $v0, 5
    syscall
    addi $t7, $v0, 0 # item
    blt $v0, 1, inv
    bgt $v0, 5, inv
    beq $v0, 4, cart
    beq $v0, 5, searchByShop
    la $a0, prompt22
    li $v0, 4
    syscall
    li $v0, 5
    syscall
    addi $t6, $v0, 0 # n
    beq $t7, 1, calculate_item1
    beq $t7, 2, calculate_item2
    j calculate_item3

calculate_item1:
    lw $t5, 16($t0)     # Load first price      # load Mole[shop_choice].first into $t1
    mul $t5, $t5, $t6    # multiply n with Mole[shop_choice].first
    add $s4, $s4, $t5
    lw $a0, 4($t0) # item
    lw $a1, 16($t0) # price
    move $a2, $t6      # n
    jal push_back
    j printu
calculate_item2:
    lw $t5, 20($t0)     # Load second price
    mul $t5, $t5, $t6    # multiply n with Mole[shop_choice].first
    add $s4, $s4, $t5
    lw $a0, 8($t0)
    lw $a1, 20($t0)     # Load second price
    move $a2, $t6
    jal push_back
    j printu
calculate_item3:
    lw $t5, 24($t0)     # Load third price
    mul $t5, $t5, $t6    # multiply n with Mole[shop_choice].first
    add $s4, $s4, $t5
    lw $a0, 12($t0)
    lw $a1, 24($t0)     # Load second price
    move $a2, $t6
    jal push_back
    j printu
    inv:
    la $a0, prompt4
    li $v0, 4
    syscall
    j printu
    jr $ra

```

SearchByItem Function

Description

this code snippet allows users to search for items by name, displays item details, validates user input, and directs the program flow based on the user's choices within the shopping system.

Algorithm

- displaying a prompt asking the user to choose an item using syscall 4 to print a string.
- Item Display Loop
 1. enters a loop labeled Print_loop that iterates through the item data stored in the Mole variable.
 2. For each item, it loads the item name and price from memory and prints them to the console using syscall 4 for strings and syscall 1 for integers.
 3. prints a newline character after each item to separate them.
- prompts the user to enter their choice using syscall 4 to print a string and syscall 5 to read the user's input, which is stored in register \$t0.
- checks if the user's choice is valid by comparing it with the minimum and maximum valid choices (1 and 11, respectively).
- If the choice is invalid, it displays an error message and jumps back to the searchByItem label to prompt the user again.
- If the user chooses option 10, it jumps to the cart label to handle the selected item.
- If the user chooses option 11, it jumps to the StartShopping label to restart the shopping process.
- If the user chooses any other option, it jumps to the item_selected label to handle the selected item.

The Code

```
searchByItem:
    # Print the item choices
    li $v0, 4
    la $a0, choose_item_str
    syscall

print_shop:
    la $t0, Mole          # Load address of Mole
    li $t1, 1             # Index for Mole[1]

Print_loop:
    lw $a0, 4($t0)         # Load item1
    li $v0, 4
    syscall
    li $v0, 4
    la $a0, Space
    syscall
    lw $a0, 16($t0)
    li $v0, 1
    syscall

    li $v0, 4
    la $a0, newline
    syscall

    lw $a0, 8($t0)         # Load item2
    li $v0, 4
    syscall
    li $v0, 4
    la $a0, Space
    syscall
    lw $a0, 20($t0)
    li $v0, 1
    syscall
    li $v0, 4
    la $a0, newline
    syscall

    lw $a0, 12($t0)        # Load item3
    li $v0, 4
    syscall
    li $v0, 4

    li $v0, 4
    syscall

    li $v0, 4
    syscall

    li $v0, 4
    syscall

    # Print newline
    li $v0, 4
    la $a0, newline
    syscall

    # Move to next Mole element
    addi $t0, $t0, 32
    addi $t1, $t1, 1
    bne $t1, 4, Print_loop # Print data for Mole[1], Mole[2], and Mole[3]

    li $v0, 4
    la $a0, cart_exit_str
    syscall
    li $v0, 4
    la $a0, choice_str
    syscall

    # Read the user's choice
    li $v0, 5
    syscall
    move $t0, $v0
    # Handle the user's choice
    bne $t0, 1, invalid_choice
    bne $t0, 11, invalid_choice
    beq $t0, 10, cart
    beq $t0, 11, StartShopping
    move $a0, $t0
    jal item_selected
    j searchByItem
```

The OutPut

Please choose the item

1)puma sneaker V2 --> 3499
2)mens dryflex --> 2999
3)one8x puma --> 5999

4)US polo Tshirt --> 999
5)Shoes --> 2000
6)shirt --> 1499

7)Mens Watch --> 2000
8)mens solid jacket --> 2239
9)casual T-shirt --> 799

10) Cart
11) Exit
Please Enter your choice : 6
Please Enter the Count of 4

Item_selected Function

Description

this function snippet handles the processing of user-selected items, calculates the total cost based on quantity and price, updates the total amount spent, and manages the flow of the shopping process within the program.

Algorithm

- determining the item type based on the user's selection and the shop type.
- checks the item range and sets the shop type accordingly (1, 2, or 3) for further processing.
- Item Processing
 - For item type 1, it prompts the user to enter the quantity, calculates the total price based on the quantity and price, and updates the total cost.
 - For item type 2 and 3, it follows a similar process of quantity input, price calculation, total cost update, and pushing back the calculated price.
- loads the appropriate prices based on the shop type and item type to calculate the total cost for each item by multiplying the quantity by the price to determine the total cost and updates the running total.
- prompts the user to enter the quantity, calculates the total cost, and updates the total amount spent.
- handles invalid item selections by displaying an error message and prompting the user again.

The Code

```
item_selected:
    la $s0, Mole          # Load address of Mole
    addi $t0, $s0, 0

    # Check if item is in range 1-3
    bge $a0, 1, check_shop1
    j invalid_item

check_shop1:
    ble $a0, 3, set_shop1
    bge $a0, 4, check_shop2
    j invalid_item

check_shop2:
    ble $a0, 6, set_shop2
    j set_shop3

set_shop1:
    li $s7, 1
    j process_item

set_shop2:
    li $s7, 2
    j process_item

set_shop3:
    li $s7, 3
    j process_item

process_item:
    rem $t1, $a0, 3 # Calculate item type (1
    beq $t1, $zero, item3
    beq $t1, 2, item2
    # If item type is 1
item1:
    li $v0, 4
    la $a0, prompt22
    syscall
    li $v0, 5
    syscall

    move $t2, $v0 # Store quantity in $t2

    # Load appropriate prices based on shop type
    beq $s7, 1, Price1
    beq $s7, 2, Price2

Price2:
    addi $t0, $t0, 32
Price1:

    lw $t3, 16($t0) # Load first price
    mul $t4, $t2, $t3 # Calculate total for item 1
    add $s4, $s4, $t4 # Update total

    # Push back calculated price
    lw $a0, 4($t0)
    lw $a1, 16($t0) # Load second price
    move $a2, $t2
    jal push_back
    j searchByItem

    # If item type is 2
item2:
    li $v0, 4
    la $a0, prompt22
    syscall
    li $v0, 5
    syscall
    move $t2, $v0 # Store quantity in $t2

    # Load appropriate prices based on shop type
    beq $s7, 1, Price12
    beq $s7, 2, Price22

Price22:
    addi $t0, $t0, 32
Price12:

    lw $t3, 20($t0) # Load first price
    mul $t4, $t2, $t3 # Calculate total for item 2
    add $s4, $s4, $t4 # Update total

    # Push back calculated price
    lw $a0, 4($t0)
    lw $a1, 24($t0) # Load second price
    move $a2, $t2
    jal push_back
    j searchByItem

    # If item type is 3
item3:
    li $v0, 4
    la $a0, prompt22
    syscall
    li $v0, 5
    syscall
    move $t2, $v0 # Store quantity in $t2

    # Load appropriate prices based on shop type
    beq $s7, 1, Price13
    beq $s7, 2, Price23

Price23:
    addi $t0, $t0, 32
Price13:

    lw $t3, 24($t0) # Load first price
    mul $t4, $t2, $t3 # Calculate total for item 3
    add $s4, $s4, $t4 # Update total

    # Push back calculated price
    lw $a0, 4($t0)
    lw $a1, 24($t0) # Load second price
    move $a2, $t2
    jal push_back
    j searchByItem

invalid_item:
    li $v0, 4
    la $a0, prompt4
    syscall
    j item_selected
```

Purchase Function

Description

this function appears to be initializing a vector data structure with a size of 0, storing the address of a symbol called p in the \$v1 register, and then returning to the caller. The function also saves and restores the return address on the stack to ensure proper control flow.

Algorithm

- adjusts the stack pointer by subtracting 8 from it. This creates space on the stack for two 4-byte values (in this case, the return address and potentially other data).
- stores the return address (\$ra register) at the memory location pointed to by \$sp + 4. This saves the return address on the stack so that the function can return to the correct location later.
- immediate value 0 into the \$a0 register. The \$a0 register is often used to pass the first argument to a function.
- jump and link" instruction that jumps to the Vector_constructor_int_n function and saves the return address in the \$ra register. This likely initializes a vector data structure with the size specified in the \$a0 register (which is 0 in this case).
- loads the address of the symbol p into the \$t9 register.
- copies the value in \$t9 (the address of p) into the \$v1 register.
- loads the immediate value 0 into the \$t8 register.
- loads the return address from the memory location pointed to by \$sp + 4 into the \$ra register.
- adjusts the stack pointer by adding 8 to it. This effectively removes the space that was allocated on the stack earlier and return.

The Code

```
purchase:
addi $sp, $sp, -8      # Adjust stack pointer
sw $ra, 4($sp)         # Save return address
# Initialize the vector
li $a0, 0
jal Vector_constructor_int_n
la $t9, p              # Load address of p
move $v1, $t9
li $t8, 0              # Index for p[0]
lw $ra, 4($sp)         # Restore return address
addi $sp, $sp, 8       # Restore stack pointer
jr $ra
```

Vector_constructor_int_n Function

Description

this function snippet dynamically allocates memory for a vector, initializes it with a specified size, stores necessary information within the allocated memory block, and then returns after completing the initialization process.

Algorithm

- adjusting the stack pointer to allocate space for two 4-byte values by subtracting 8 from the stack pointer and saving the return address on the stack.
- calculates the size of the vector by left-shifting the value in register \$a0 by 2 bits (equivalent to multiplying by 4) and uses the sbrk system call (syscall 9) to dynamically allocate memory for the vector.
- address of the allocated memory block is stored in register \$t1, and the address itself and the size of the vector are stored within the allocated memory block for future reference.
- restores the return address from the stack, adjusts the stack pointer back to its original position, and returns to the calling function using the return address stored earlier.

The Code

```
/vector_constructor_int_n:
```

```
addi $sp, $sp, -8
sw $ra, 4($sp)
```

```
sll $t0, $a0, 2
li $v0, 9
move $a0, $t0
syscall
move $t1, $v0
```

```
sw $t1, 0($t1)
sw $a0, 4($t1)
```

```
lw $ra, 4($sp)
addi $sp, $sp, 8
jr $ra
```

Puch_back Function

Description

This function snippet implements a function that adds an item to the end of a vector, updating the vector's structure and maintaining the count of structs.

Algorithm

- adjusting the stack pointer (\$sp) to make space for the function's local variables.
- saves the return address (\$ra) on the stack to preserve it for later use.
- initializes three local variables: item (\$s5), price (\$s6), and N (\$s7), which are passed as arguments to the function.
- updates the vector by adding a new struct at the end. It does this by:
 1. Storing the item, price, and N values in the new struct.
 2. Incrementing the pointer to the next node (\$t9) by 12 bytes (the size of the struct).
 3. Incrementing the count of struct (\$t8) by 1.
- restores the return address (\$ra) from the stack and adjusts the stack pointer (\$sp) to its original value.
- jumps to the return address using the jr instruction, effectively returning from the function.

The Code

```
push_back:
addi $sp, $sp, -8      # Adjust stack pointer
sw $ra, 4($sp)         # Save return address
move $s5,$a0 #item
move $s6,$a1 #price
move $s7,$a2 # N

sw $s5, 0($t9)
sw $s6, 4($t9)
sw $s7, 8($t9)
addi $t9,$t9,12
addi $t8,$t8,1

lw $ra, 4($sp)         # Restore return address
addi $sp, $sp, 8       # Restore stack pointer
jr $ra
```

Cart Function

Description

this function snippet manages the user's shopping cart by displaying the cart contents, allowing item removal, and calculating and displaying the total order cost. It provides a user-friendly interface for managing items in the shopping cart within the program.

Algorithm

- displaying a header for the shopping cart using syscall 4 to print a string.
- prompts the user to enter their name to associate with the cart using syscall 4 and 5.
- calls a function print_push to print the items in the cart.
- Displays a prompt to remove an item from the cart and reads the user's choice using syscall 5.
- Item Removal
 1. If the user chooses to remove an item (option 1), it calls the pop_back function to remove the last item from the cart.
 2. If the user does not choose to remove an item, it proceeds to display the total order cost.
- Displays a string indicating the total order cost using syscall 4 to print a string.
- Prints the total order cost stored in register \$s4 using syscall 1 for integers.

The Code

```
cart:
    li $v0, 4
    la $a0, cart_header_str
    syscall
    li $v0, 4
    la $a0, prompt16
    syscall
    la $a0, t_name
    syscall

jal print_push
    li $v0, 4
    la $a0, remove_item_str
    syscall
    li $v0, 5
    syscall
    move $t0, $v0

    beq $t0, 1, pop_back
    END_Cart:

    li $v0, 4
    la $a0, total_order_str
    syscall
    li $v0, 1
    move $a0, $s4
    syscall
```

The OutPut

```
*****Cart*****
```

```
Welcome habiba
```

Element	price	Amount
2)mens dryflex	--> 2999	--> 3
1)puma sneaker V2	--> 3499	--> 5
3)one8x puma	--> 5999	--> 6

```
Your Total Order Amount is : 62486
```

Print_push Function

Description

Function that prepares to print the contents of a purchase vector. It manages the stack, checks for an empty vector, initializes variables, and displaying item details and the total order cost to the user.

Algorithm

- adjusting the stack pointer (\$sp) to make space for the function's local variables.
- saves the return address (\$ra) on the stack to preserve it for later use.
- checks if the count of structs (\$t8) is zero. If the vector is empty, it jumps to the end label to exit the function.
- initializes a temporary variable \$t1 to 0.
- displays a string indicating the cart amount using syscall 4 to print a string.
- enters a loop labeled item_print to iterate through the items in the vector.
 1. For each item, it loads and prints the item name, price, and quantity using syscalls 4 and 1.
 2. increments the pointer to the next node and the loop counter to move to the next item.
- displays a string indicating the total order cost using syscall 4
- prints the total order cost stored in register \$s4 using syscall 1 for integers.
- estores the return address (\$ra) from the stack and adjusts the stack pointer (\$sp) to its original value.
- jumps to the return address using the jr instruction, effectively returning from the function.

The Code

```
print_push:
addi $sp, $sp, -8      # Adjust stack pointer
sw $ra, 4($sp)
beq $t8, $zero, end
li $t1, 0
    li $v0, 4
    la $a0, cart_amount_str
    syscall

item_print:
beq $t1, $t8, end

lw $a0, 0($v1)
li $v0, 4
syscall
    li $v0, 4
    la $a0, Space
    syscall

lw $t2, 4($v1)
move $a0, $t2
li $v0, 1
syscall
    li $v0, 4
    la $a0, Space
    syscall

lw $t2, 8($v1)
move $a0, $t2
li $v0, 1
syscall

    li $v0, 4
    la $a0, newline
    syscall
addi $v1, $v1, 12
addi $t1, $t1, 1
j item_print
```

The Output

```
end:
li $v0, 4
la $a0, total_order_str
syscall
li $v0, 1
move $a0, $s4
syscall
    lw $ra, 4($sp)      # Restore return address
    addi $sp, $sp, 8    # Restore stack pointer
    jr $ra
```

*****Cart*****

Welcome habiba

Element	price	Amount
2)mens dryflex	--> 2999	--> 3
1)puma sneaker V2	--> 3499	--> 5
3)one8x puma	--> 5999	--> 6

Your Total Order Amount is : 62486

Pop_back Function

Description

This function handle removing the last item, updating the total order cost.

Algorithm

- removes the last element from the vector by moving the pointer back by 12 bytes (size of each element).
- calculates the total cost of the removed item and updates the total order cost.
- clears the removed item by setting all values to 0.
- decrements the count of elements in the vector.
- displays a message indicating that the last item has been removed.
- jumps to the END_Cart label to continue the program flow.

The Code

```
pop_back:

addi $v1,$v1,-12
lw $t0, 4($v1)
lw $t1, 8($v1)
mul $t1,$t1,$t0
sub $s4,$s4,$t1
sw $0,0($v1)
sw $0, 4($v1)
sw $0, 8($v1)
addi $v1,$v1,-12
addi $t8,$t8,-1

li $v0, 4
la $a0,Lastisremoved
syscall

j END_Cart
```

The Output

```
*****Cart*****

Welcome aya

Amount          price          Element
1)puma sneaker V2    --> 3499          --> 2
2)mens dryflex    --> 2999          --> 1

Your Total Order Amount is : 9997

Do you want to remove the last item from the cart? (y=1/n=0) : 1

Last item removed from the cart.

Your Total Order Amount is : 6998
```

Clear

Order function

Description

This function snippet handles the order confirmation and cancellation process within the shopping system. It prompts the user for their choice, processes confirmed orders, and allows for order cancellation with the option to exit the program.

Algorithm

- displaying a prompt asking the user to confirm the order using syscall 4 to print a string.
- reads the user's choice using syscall 5 and stores it in register \$t0.
- If the user chooses option 1, it jumps to the order_confirmed label to process the confirmed order.
- If the user chooses option 0, it jumps to the cancel_order label to handle the order cancellation.
- If the user enters an invalid choice, it displays an error message and jumps back to the Order label to prompt the user again.
- Confirmed Order Processing
 1. displays a confirmation message using syscall 4.
 2. prints the user's name stored in t_name using syscall 4.

3. displays an order confirmation message and jumps to the exit_program label to terminate the program.
- Order Cancellation
 1. sets the total order cost to 0 by storing 0 in register \$s4.
 2. displays a prompt asking the user to cancel the order or exit the program using syscall 4.
 3. reads the user's choice using syscall 5 and stores it in register \$t3.
 - If the user chooses option 1, it jumps to the Cancel_order label to process the order cancellation.
 - If the user chooses option 0, it jumps to the exit_cancel_or_exit label to exit the program.
 - If the user enters an invalid choice, it sets the total order cost to 0 and jumps to the exit_program label to terminate the program.
 - In the Cancel_order label, it displays a confirmation message using syscall 4.
 - calls a function Vector_Clear to clear the shopping cart.
 - calls a function print_push to print the updated cart contents.

The Code

```
Order:
    li $v0, 4
    la $a0, prompt20
    syscall

    li $v0, 5
    syscall
    move $t0, $v0

    beq $t0, 1, order_confirmed

    beq $t0, 0, cancel_order

    li $v0, 4
    la $a0, prompt4
    syscall
    j Order

order_confirmed:
    li $v0, 4
    la $a0, prompt21
    syscall

    li $v0, 4
    la $a0, t_name
    syscall

    li $v0, 4
    la $a0, order_confirmation_message
    syscall
    j exit_program

cancel_order:
    li $s4, 0

    li $v0, 4
    la $a0, cancel_or_exit_options
    syscall

    li $v0, 5
    syscall
    move $t3, $v0

    li $t4, 1
    beq $t3, $t4, Cancel_order
    bne $t3, $zero, exit_cancel_or_exit

Cancel_order:
    li $v0, 4
    la $a0, prompt21
    syscall

    jal Vector_Clear

    jal print_push

exit_cancel_or_exit:
    li $s4, 0
    j exit_program
```

The Output

```
Do You wish to order (y=1/n=0) : 1

Thank You aya
for your Order
Your item is on the way. Welcome again

Thanks for Visiting!!

-- program is finished running --
```

Vector_clear function

Description

This function handle clearing all items from a, updating the total order cost to 0.

Algorithm

- adjusts the stack pointer to make space for local variables and saves the return address on the stack.
- checks if the vector is empty by comparing the count of elements (\$t8) to zero. If the vector is empty, it jumps to the endC label to exit the function.
- initializes a temporary variable \$t1 to 0.
- Item Clear Loop
 1. enters a loop labeled item_clear to iterate through the items in the vector.
 2. each item, it clears the contents by setting all values to 0.
 3. resets the count of elements to 0 and returns from the function.

The Code

```
Vector_Clear:
addi $sp, $sp, -8    # Adjust stack pointer
sw $ra, 4($sp)

beq $t8, $zero, endC
li $t1, 0

item_clear:
beq $t1, $t8, endC

sw $0, 0($v1)
sw $0, 4($v1)
sw $0, 8($v1)

addi $v1, $v1, -12
addi $t1, $t1, 1
j item_clear
endC:
move $t8, $0
lw $ra, 4($sp)      # Restore return address
addi $sp, $sp, 8
jr $ra
```

The Output

Do you want to remove the last item from the cart? (y=1/n=0) : 0

Your Total Order Amount is : 62486

Exit_Program function

Description

when the program reaches the `exit_program` label, it prints a message stored in `prompt3` using `syscall 4`, and then exits the program using `syscall 10`.

Algorithm

- loads the value 4 into register `$v0`, which is the `syscall` code for printing a string.
- loads the address of `prompt3` into register `$a0`, which is the argument for the string to be printed.
- executes the `syscall` using `syscall`.
- loads the value 10 into register `$v0`, which is the `syscall` code for exiting the program.
- Finally, it executes the `syscall` again to

The Code

```
exit_program:  
li $v0, 4  
la $a0, prompt3  
syscall  
li $v0 10  
syscall
```

The Output

```
Thanks for Visiting!!
```

```
-- program is finished running --
```

Task Assignment Table

Shopping Cart Using Vector DS				Project CO	Team 1	
Presentation	Report	Mips Code	CPP Code	Searching about DS & Idea & Vector		
25%	25%	25%	25%	25%	Aya Mohamed Sayed	
25%	25%	25%	25%	25%	Bosina Saad	
25%	25%	25%	25%	25%	Habiba Ahmed	
25%	25%	25%	25%	25%	Nadia Kamel	

Task Assignment Table

Shopping Cart Using Vector DS				Project CO	Team 1	
Start Shopping Function	Login	Sign-Up	Main Func.	Data Segment		
25%	25%	25%	25%	25%	Aya Mohamed Sayed	
25%	25%	25%	25%	25%	Bosina Saad	
25%	25%	25%	25%	25%	Habiba Ahmed	
25%	25%	25%	25%	25%	Nadia Kamel	

Task Assignment Table

Shopping Cart Using Vector DS					Project CO
selected item	selectedShop	Shop initialization	SearchBy Shop	SearchBy item	Team 1
25%	25%	25%	25%	25%	Aya Mohamed Sayed
25%	25%	25%	25%	25%	Bosina Saad
25%	25%	25%	25%	25%	Habiba Ahmed
25%	25%	25%	25%	25%	Nadia Kamel

Task Assignment Table

Shopping Cart Using Vector DS					Project CO
print_push	VectorClass	Order	Exit_program	Cart	Team 1
25%	25%	25%	25%	25%	Aya Mohamed Sayed
25%	25%	25%	25%	25%	Bosina Saad
25%	25%	25%	25%	25%	Habiba Ahmed
25%	25%	25%	25%	25%	Nadia Kamel

**ELECTRICAL ENGINEERING
DEPARTMENT
ELE232 – COURSE PROJECT (TERM 24-
02)**



Course Project cover page

S#	Student Name	Edu Email	Student ID	Marks			
				Report & Slides (30)	Implementation (50)	Presentation (20)	Total (100)
1	Aya Mohamed	aya403175@feng.bu.edu.eg	9				
2	Habiba Ahmed	habiba402985@feng.bu.eg	13				
3	Bosina Saad	bothaina403065@feng.bu.edu.eg	10				
4	Nadia Kamel	nadia402984@feng.bu.edu.eg	69				
5							

Date handed in: / / 2024

